FUNCTIONAL REQUIREMENTS DOCUMENT

WinCapture

FRD v1.0 | Based on WinCapture project documentation | 21 October 2026

1. Document Control

Item	Details
Document	Functional Requirements Document (FRD)
Project	WinCapture
Version	1.0
Status	Draft / For Project Review
Prepared from	WinCapture documentation bundle and consolidated source-code snapshot
Document date	21 October 2026

2. Purpose

This document describes the main user-facing functions and expected system behaviour of WinCapture. It brings together the supplied project documentation and code snapshot in one place so that the application can be reviewed, tested and maintained against a consistent baseline.

The document covers Microsoft Entra ID sign-in, file handling, albums, album sharing, administrator operations, the React and Angular frontend arrangement, and the API and storage responsibilities behind those features. It is intended to be used alongside the repository API reference and the source files when implementation details need to be checked.

3. Project Reference and System Overview

WinCapture is a browser-based application for uploading and managing files, grouping files into albums, and sharing albums with other registered users. A shared album can grant viewing and downloading separately. Administrators have additional views for managing files and albums across the application.

Microsoft Entra ID handles sign-in. After the API validates the delegated access token, the application maps the Microsoft identity to a WinCapture user and applies the user role, ownership rules and album permissions. File metadata is stored in SQL Server; file content is stored in Azure Blob Storage.

The main interface is a React and TypeScript single-page application. The Albums listing is implemented as an Angular page served under /angular on the same frontend origin. The Angular page reuses the existing MSAL browser session rather than running a second sign-in flow.

4. Functional Scope

- Sign in through Microsoft Entra ID and load the current WinCapture user and role.
- Upload, list, search, preview, download, replace, favorite and delete supported files.
- Create, list, rename and delete albums, subject to ownership and administrator rules.
- Add files to albums and perform album-scoped file operations.
- Share albums with existing users and grant or update view and download permissions separately.
- Browse albums that the user owns or can access through sharing.
- Provide administrator views for application-wide files and albums, as well as the administrator's own resources.
- Apply server-side pagination, validation, authorization and consistent API error responses.
- Keep metadata and file content operations coordinated across SQL Server and Azure Blob Storage.

The supplied WinCapture materials do not describe blog post publishing, reader comments or comment moderation. Those functions belong to the separate Blog Platform FRD and are not part of this WinCapture scope.

5. User Roles and Permissions

WinCapture has two application roles: User and Admin. Album ownership and album membership add resource-specific permissions; they are not separate application roles. A person may own one album and also be a member of another.

Capability	User / Owner	Shared member	Admin
Sign in and view own account	Yes, with a valid Entra identity and assigned role	Yes, with a valid Entra identity and assigned role	Yes
Manage own standalone files	Yes, for files they own and within album rules	No, unless they also own the file	Yes, subject to service checks
Create and manage own albums	Yes, as album owner	Not by membership alone	Yes, including admin workflows
View shared album content	If owner or granted CanView	Only when CanView is granted	Subject to admin service rules
Download shared album content	If owner or granted CanDownload	Only when CanDownload is granted	Subject to admin service rules
Add or change album members	Owner can manage membership	No, membership alone does not grant this	Administrative management is available
View application-wide file/album lists	No	No	Yes, through admin endpoints

A visible button or frontend route is not an authorization check. The API and service layer must verify role, ownership and album membership for each protected action.

6. Functional Requirements

ID	Function	Requirement	Expected system behaviour
FR-001	Authentication	The system shall use Microsoft Entra ID for user sign-in and require a valid delegated API access token for protected API calls.	Invalid or missing authentication is rejected.
FR-002	User mapping	The system shall map the authenticated Microsoft identity to a WinCapture user record and application role.	The API returns the mapped current-user details and role.
FR-003	File upload	A user shall be able to upload supported files after file type, content and size validation.	Valid content is stored; invalid uploads are rejected with a clear response.
FR-004	File gallery	A user shall be able to retrieve accessible files, including relevant favorite and file metadata.	Only files permitted for the caller are returned.
FR-005	File preview/download	The system shall provide separate preview and download behaviour where the file workflow supports it.	The API checks permission before returning file content.
FR-006	Replace/delete file	A permitted user shall be able to replace or delete a file.	Ownership, role and album rules are checked before changes are saved.
FR-007	Favorites	A user shall be able to list, add and remove favorites for files they can access.	The favorite relationship is unique per user and file.
FR-008	Album management	A permitted user shall be able to create, list, rename and delete albums.	Album access and ownership rules are applied to each operation.
FR-009	Album sharing	An album owner or permitted administrator shall be able to add members and change or revoke their access.	The stored grant records CanView and CanDownload separately.

ID	Function	Requirement	Expected system behaviour
FR-010	Album files	A permitted user shall be able to list, upload, preview, download, replace or remove files in an album.	The API checks album access and file-to-album association.
FR-011	Admin views	An administrator shall be able to retrieve application-wide file and album listings and their own resources.	Admin endpoints apply the assigned Admin role.
FR-012	Search and pagination	Listings that support search, filtering or paging shall return only the requested results and relevant paging metadata.	Paging is handled by the API/repository rather than by loading the full result set.
FR-013	Validation	Required inputs, file types, file size and resource identifiers shall be validated before a change is persisted.	Invalid requests do not create a valid-looking partial record.
FR-014	Storage consistency	The system shall coordinate file content changes with metadata changes and attempt cleanup when a later step fails.	Cleanup failures are logged and can be investigated.
FR-015	Authorization	Enforce role, ownership and album membership rules at the API.	UI checks do not replace API authorization.
FR-016	Error handling	Return handled failures in a consistent JSON structure and HTTP status.	Log unexpected errors and return a generic response.

7. Detailed Functional Specifications

7.1 Authentication and current-user context

- The React application validates its public environment configuration before starting MSAL. If required values are missing or startup fails, it displays a startup error instead of mounting the application.
- MSAL requests a delegated access token for the configured WinCapture API scope. The API validates the bearer token and the required access_as_user scope.
- The claims transformation resolves the Microsoft Entra object ID, email and assigned role to a WinCapture user. It adds the local WinCapture user ID used by controllers and services.

- The current-user endpoint returns the local user ID, name, email, role and authentication type.
- The Angular Albums page reads the cached account and silently acquires a token. It relies on the existing React-origin session and must not introduce an independent login flow.

7.2 File management

The file workflow begins in the React file pages and feature components. API helpers and feature hooks make authenticated requests, maintain list/query state, and coordinate previews and transfers. The backend file service validates input, checks permissions and coordinates file metadata with blob storage.

- Uploaded file metadata includes the original name, generated storage name, content type, size, uploader, optional album and upload timestamp.
- The supplied validator source sets a maximum file size of 10 MB and checks extension, declared content type and file structure. The allowlist in the snapshot includes common image, document, video, audio, text/CSV and ZIP formats.
- Preview and download are separate operations. Album membership and CanView / CanDownload determine which action is allowed.
- During replacement, the new blob is written and metadata is updated before the previous blob is removed. If metadata persistence fails after a new upload, the service attempts to delete the new blob.
- Deletion removes metadata and attempts storage cleanup. If cleanup does not complete, the failure must be logged and handled through follow-up reconciliation.

7.3 Albums and sharing

Albums group files and can be shared with registered WinCapture users. The React application owns album detail, sharing and album file actions. The Angular application handles the Albums listing page, including its list state and album creation flow.

- The album list can include albums owned by the user and albums shared with them, subject to access rules. Separate queries are available for owned albums and administrator listings.
- Album owners and administrators can perform management operations as allowed by the service layer.
- A member grant stores CanView and CanDownload separhiately. Permission to preview or see album content does not automatically imply permission to download it.
- The API checks the album and file association before permitting album-scoped replace, preview, download or deletion.
- Deleting an album also affects its associated metadata and can require blob cleanup for files no longer retained elsewhere.

7.4 Favorites and administration

Favorites are stored as a relationship between a user and a file. The database key prevents duplicate favorite records for the same user/file pair. A user can retrieve their favorites and add or remove a favorite through the file API.

The Admin area offers wider file and album listings, including all-user views and separate “mine” endpoints. These endpoints must remain protected by the WinCapture.Admin application role. Resource-level checks still apply to operations that depend on ownership or album rules.

7.5 Validation and pagination

The API and service layer validate incoming operations. For uploads, the validator checks that a file is supplied, the extension and content type are supported, the file is within the configured limit and the content matches the expected file structure. Frontend schemas and types help display data safely, but are not a substitute for server validation.

8. Frontend Architecture and Navigation

The two frontends are separate implementations that share the same browser origin and backend API. React remains the main application shell. Angular is used for the Albums listing page only in the documented baseline; album details, file management and administration remain in React.

8.1 Angular Albums page - low-level frontend flow

The following flow shows the Angular page from navigation through token reuse, API requests, service checks and list rendering. The Angular implementation files are not included in the supplied consolidated code snapshot, so this diagram follows the accompanying project documentation.

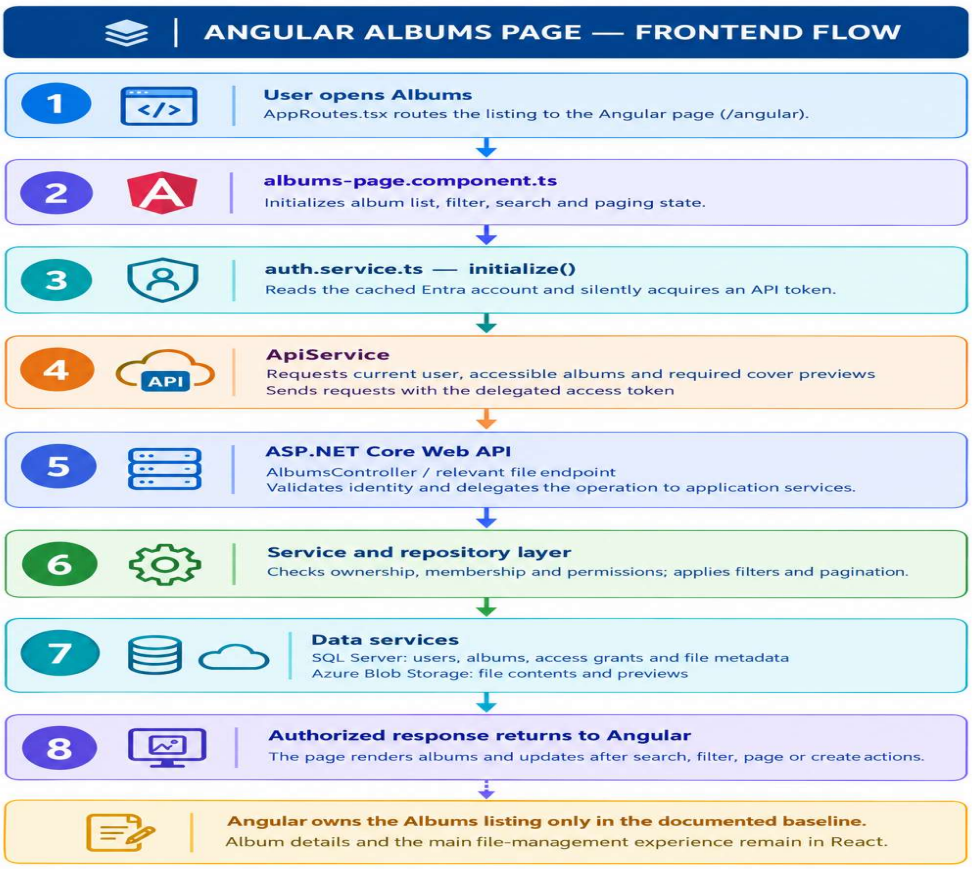


Figure 1. Low-level flow for the Angular Albums listing page.

8.2 How React and Angular work together

React and Angular are not one component tree. The main React SPA provides the overall app experience and the MSAL authentication context. The Angular page is mounted under /angular on the same origin and reuses the cached Entra account. Both applications send authenticated API calls to the same ASP.NET Core backend.

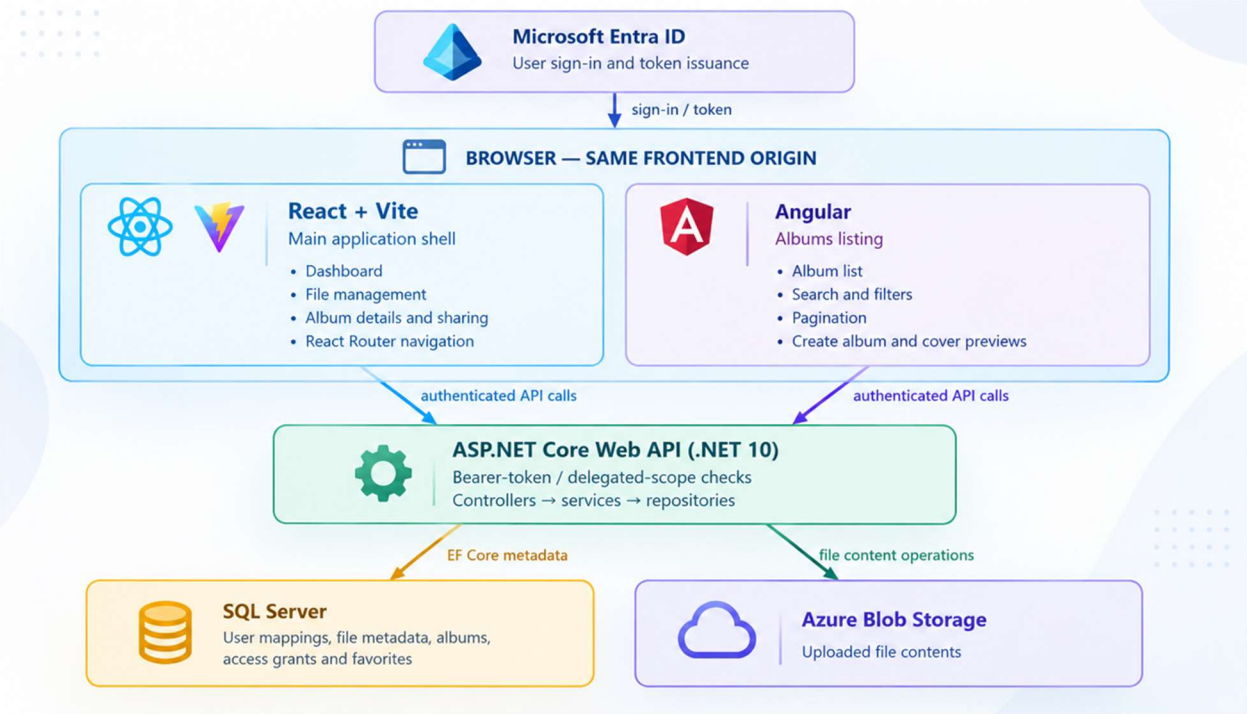


Figure 2. Frontend and backend relationship in the documented WinCapture baseline.

9. Authentication Workflow

WinCapture does not issue a second application JWT after Microsoft sign-in. The browser obtains a delegated access token from Microsoft Entra ID and sends it to the API. The API validates the token and scope, maps the identity to a local WinCapture user, and applies role and resource permissions to the requested operation.

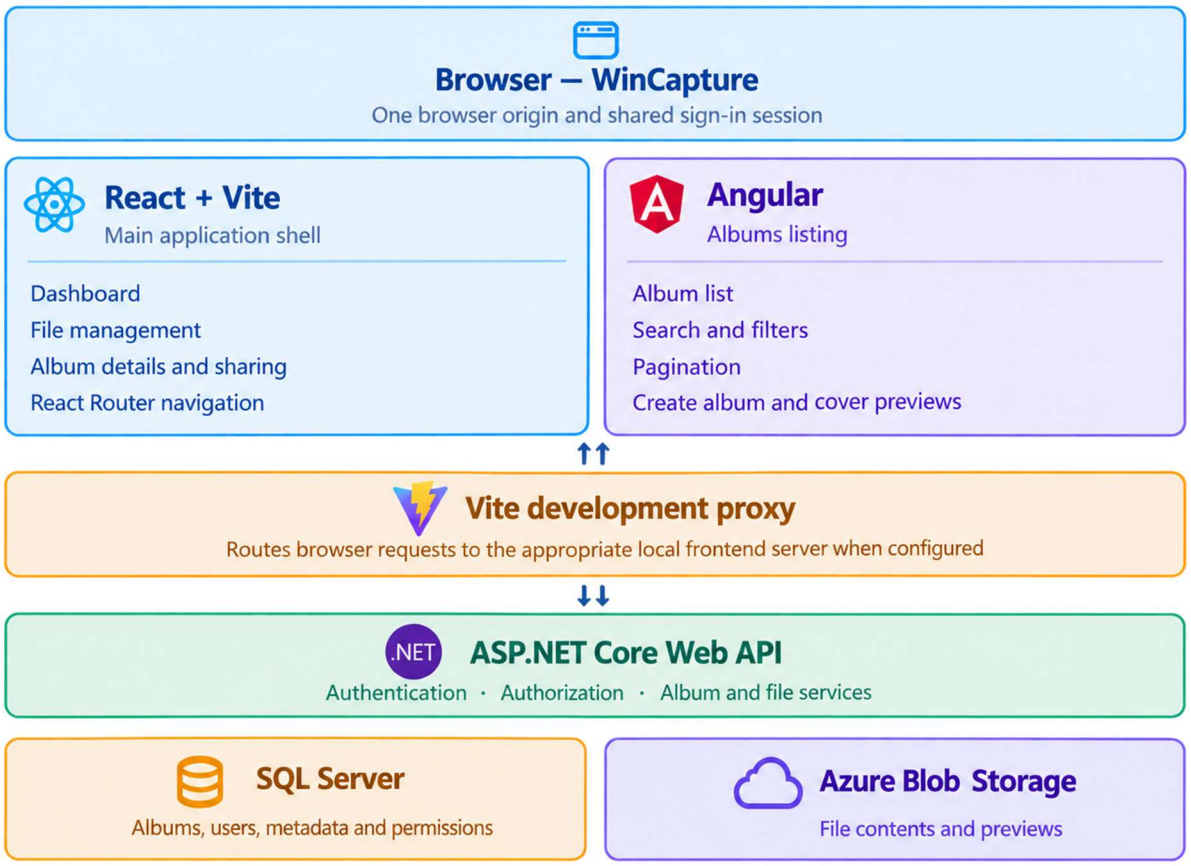


Figure 3. End-to-end authentication and authorization flow.

10. Data and Validation Rules

Entity	Purpose	Key relationships / rules
User	Maps a Microsoft Entra identity to a WinCapture user.	Email and Entra object ID are unique; role is User or Admin.
FileMetadata	File properties and storage-name record.	Uploader required; album optional; stored name is unique.
Album	Named collection of files with an owner.	Owner/updater reference users; deletion cascades to related file metadata.

Entity	Purpose	Key relationships / rules
AlbumAccess	Album membership grant.	Unique album/user pair; stores view/download flags and grant audit fields.
Favorite	User/file favorite link.	Composite UserId/FileId key; removed with the user or file.

EF Core mappings and migrations define the SQL Server schema. Review the separate database setup script when changing the model.

11. API Functional Requirements

The API is the system boundary for identity checks, resource authorization, metadata changes and file content operations. Controllers bind request data and delegate workflow rules to the service layer. The API reference in the repository remains the source of truth for exact request and response contracts.

API area	Representative route(s)	Purpose
Authentication	GET /api/auth/me	Return the current mapped WinCapture identity and role.
Files	POST/GET /api/files; GET/PUT/DELETE /api/files/{id}	Upload, list, retrieve, replace and delete files.
Favorites	GET /api/files/favorites; PUT/DELETE /api/files/{id}/favorite	List and change file favorites.
Download	GET /api/files/{id}/download	Return authorized file content with a download name.
Albums	POST/GET /api/albums; GET /api/albums/mine; GET/PUT/DELETE /api/albums/{id}	Create and retrieve accessible albums; manage owned albums.
Album files	/api/albums/{id}/files and /api/albums/{id}/files/{fileId} route family	Upload, list, replace, retrieve, preview, download and delete album files.
Album members	/api/albums/{id}/members route family	List members, grant access, update permissions and remove members.
Administration	GET /api/admin/files; GET /api/admin/albums; “mine” variants	Admin-wide lists and the administrator’s own resources.

12. Error and Exception Handling

Scenario	Expected response / handling
Missing, malformed or invalid input	Return a 400 Bad Request response with a handled error message.
Missing or invalid authenticated user identity	Return 401 Unauthorized; do not continue the protected operation.
Authenticated user lacks a role or resource permission	Return 403 Forbidden where access is explicitly denied.
Requested resource is missing or not available to the caller	Return 404 Not Found and do not update or delete a different resource.
Conflicting state	Return 409 Conflict where the operation cannot be completed due to the current state.
Upload exceeds the configured limit	Return 413 Payload Too Large.
Storage failure or unexpected application error	Return a generic 500 response; log the exception and request context without exposing internal details.

The shared exception middleware maps known application exception types to HTTP status codes and a JSON response containing the status and message. Unexpected server errors use a generic message; the exception is logged for diagnosis.

13. Non-Functional Requirements

Area	Requirement
Security	Validate delegated API tokens and scope. Enforce role and resource permissions in the API. Keep secrets, tokens and private configuration out of browser bundles and logs.
Performance	Use server-side paging and database queries that retrieve the requested result page. Avoid loading full result sets when a paged endpoint exists.
Reliability	Coordinate SQL metadata and blob content changes. Attempt cleanup when a later step fails and log cleanup failures for follow-up.
Maintainability	Keep controllers thin, domain checks in services, persistence in repositories and blob operations behind IStorageService.
Testability	Maintain automated backend, React and Angular tests for the workflows owned by each layer.

Area	Requirement
Usability	Show loading, validation and failure states clearly. Keep preview/download actions distinct and make album sharing permissions understandable.
Configuration	Use explicit trusted CORS origins outside development and align API scope, Entra redirect URIs, frontend origin and /angular routing for deployment.

14. Test Requirements

Test area	Coverage expected
Authentication and identity	Valid and invalid tokens/scopes, current-user response, identity mapping, assigned roles and protected endpoint behaviour.
Authorization	Positive and negative role access, owner versus non-owner actions, shared-member permissions, and separate CanView / CanDownload outcomes.
File service	Supported and rejected types, size limit, content validation, upload, replacement, download, deletion and favorite behaviour.
Storage consistency	Metadata failure after blob write, cleanup attempt, replacement cleanup and storage failure logging.
Albums and sharing	Accessible/owned/shared lists, paging, create/update/delete, member grant/update/removal and album-file association checks.
React frontend	Loading/error states, file actions, preview handling, API schemas and query/mutation behaviour.
Angular frontend	Album list state, search/filter/page interactions, creation flow, API service responses and session reuse.

The supplied project guide lists these local checks. Run them from the corresponding project directory and report any environment or configuration blocker rather than treating a skipped check as a pass.

Area	Command
Backend	dotnet test Tests\backend.Tests\WinCapture.Tests.csproj
React typecheck	npm run typecheck
React tests	npm test
React lint/build	npm run lint; npm run build

Area	Command
Angular typecheck/tests/build	npm run typecheck; npm test; npm run build

15. UI Functional Requirements

Screen / module	Expected functions
React application shell	Navigation and authenticated app layout; handles the main route structure.
Dashboard	Provide the main landing view and navigation to file and album features.
Files	Upload, browse and search files; select actions; show details; preview, download, replace, favorite and delete where permitted.
Albums listing - Angular	List accessible albums and support the documented search, filter, paging and album creation interactions.
Album details - React	Display album content; support album file operations and membership/sharing workflows.
Admin	Show the administrator's wider file and album views and applicable management actions.
Errors and dialogs	Show actionable startup errors, request errors and confirmation/details dialogs without relying on the UI to enforce security.

16. Acceptance Criteria / Functional Definition of Done

- AC-01: A registered user can sign in through Microsoft Entra ID and retrieve the mapped WinCapture user details.
- AC-02: A request without the required token or delegated API scope cannot access protected endpoints.
- AC-03: A user can upload a supported file and see it in the accessible gallery.
- AC-04: A file that fails upload validation is rejected and is not added as a successful gallery record.
- AC-05: A permitted user can preview or download a file; the API rejects an action the user is not allowed to perform.
- AC-06: A user can add and remove a favorite without creating duplicate user/file favorite records.
- AC-07: A user can create an album and see albums they own or that have been shared with them.

- AC-08: An album owner can grant or update a member's view and download permissions independently.
- AC-09: A shared member cannot view or download content when the corresponding permission is absent.
- AC-10: The Angular Albums listing works under /angular and uses the existing Entra browser session.
- AC-11: Administrator endpoints reject callers who do not hold the WinCapture.Admin role.
- AC-12: Storage and database failure scenarios are covered by automated tests, including attempted cleanup after a failed metadata save.
- AC-13: Pagination and supported search/filter operations return the requested results and relevant metadata.
- AC-14: Backend, React and Angular checks are run and the results are recorded for the release candidate.

17. Traceability Matrix

Requirement group	FRD coverage	Validation evidence
Authentication and roles	FR-001, FR-002, FR-015; Sections 7.1 and 9	Microsoft identity, current-user and API security tests.
Standalone files	FR-003 to FR-007; Section 7.2	File service, validator, storage and React file-flow tests.
Albums and members	FR-008 to FR-010; Sections 7.3 and 8	Album service, album-file service, membership permission and Angular listing tests.
Administration	FR-011; Section 7.4	Controller and role-access tests for admin endpoints.
Search and paging	FR-012; Sections 7.5 and 11	Repository paging, filtering and list-state tests.
Validation and consistency	FR-013, FR-014, FR-016; Section 12	File validator, exception middleware and storage cleanup tests.
Operational constraints	Sections 13 and 14	Build/test command results, CORS configuration and deployment checks.

18. Assumptions and Dependencies

- Microsoft Entra app registrations, tenant/client settings, API audience and the delegated access_as_user scope are configured for the target environment.
- The frontend public configuration contains only values intended for browser use. No client secret is shipped in the SPA.

- SQL Server is available and the schema is kept aligned with the EF Core migrations and database reference script.
- Azure Blob Storage connection settings and the required container are configured for the API environment.
- Trusted CORS origins, redirect URIs, the primary frontend origin and /angular routing are aligned for the deployment.
- Users to whom albums are shared already have a registered WinCapture identity that can be resolved by the application.
- The React application remains the primary frontend. Angular continues to own the Albums listing unless the project scope changes.

19. Technical Constraints / Project Stack

Component	Technology / arrangement
Primary frontend	React 19, TypeScript, Vite, React Router, TanStack Query, MSAL and Zod.
Angular frontend	Angular 21 and TypeScript; used for the Albums listing under /angular.
Backend API	ASP.NET Core Web API on .NET 10, with controllers, services and repositories.
Authentication	Microsoft Entra ID, MSAL in the browser, bearer-token validation and the delegated API scope.
Persistence	Entity Framework Core with Microsoft SQL Server. EF Core migrations represent schema changes.
File content	Azure Blob Storage behind the IStorageService abstraction.
Testing	xUnit backend tests; Vitest and Testing Library coverage for React and Angular.
API reference	Repository docs/API.md is the route and request/response contract reference.
Source control / delivery	Git repository; local setup and verification instructions are documented in the project README and project guide.

20. Documentation Baseline and Known Gaps

The current project documentation describes React as the main application shell and Angular as the Albums listing implementation under /angular. The supplied consolidated code snapshot contains 87 file sections, but it does not include the Angular source files, backend Program.cs, or test projects referenced by the documentation. Its React file layout is also older than the layout described in the documentation bundle.

For that reason, the diagrams in this document follow the supplied project documentation for the current frontend arrangement and authentication flow. The code snapshot supports details where matching files are present, including controllers, services, repositories, file validation, identity mapping and EF Core entities. Before changing a route or deployment setting, confirm it against the maintained repository and docs/API.md.

21. Review and Approval

Role	Name	Signature / comments	Date
Product / Project Owner			
Technical Lead			
QA / Reviewer			

React Workflow

